# ESP32 + PlatformIO + GitHub

## Complete VS Code Git/GitHub Setup & Push Guide

*For ESP32 PlatformIO Projects*

| | |
|---|---|
| **Environment** | Visual Studio Code + PlatformIO |
| **Hardware** | ESP32 |
| **Version Control** | Git |
| **Remote Repository** | GitHub |

# 1. What You Will Learn

- Difference between Git username, GitHub username, repository name, and project folder.
- How to configure Git inside VS Code.
- How to connect VS Code to your GitHub account.
- How to initialize Git in an existing PlatformIO project.
- How to create a suitable .gitignore file for PlatformIO.
- How to create a GitHub repository and connect it to the local project.
- How to make the first commit and push the project.
- How to update the GitHub repository after changing ESP32 code.
- How to use the VS Code Source Control GUI.

# 2. Understand the Names

Several names are used in Git and GitHub, and they are easy to confuse. The following example shows how they relate to one another.

| Term | Example / Meaning |
|---|---|
| **GitHub username** | nasirmahmood — identifies your GitHub account. |
| **Git username** | Nasir Mahmood — identifies you in local Git commits. |
| **GitHub repository name** | ESP32_Timer — the project repository on GitHub. |
| **Local project folder** | ESP32_Timer — the PlatformIO project folder on your computer. |
| **Remote URL** | https://github.com/nasirmahmood/ESP32_Timer.git — connects local Git to GitHub. |

**Key point**
Git username and GitHub username are not technically the same thing. They can be different. For a simple setup, using a consistent name and your GitHub-associated email makes the configuration easier to understand.

# 3. The Overall Relationship

```
GitHub account
    |
    +-- username: nasirmahmood
            |
            +-- repository: ESP32_Timer
                    |
                    +-- ESP32 PlatformIO project
```

# 4. Check Whether Git Is Installed

Open your ESP32 project in VS Code and select:

- Terminal → New Terminal

Run:

```
git --version
```

A successful installation may return something similar to:

```
git version 2.51.0.windows.1
```

**If Git is not recognized**
If Windows reports that 'git' is not recognized as a command, install Git for Windows, restart VS Code, and run git --version again.

## 5. Configure Your Git Username and Email

Set the name that Git will store in your commits:

```
git config --global user.name "Nasir Mahmood"
```

Set the email associated with your GitHub account:

```
git config --global user.email "your-email@example.com"
```

Verify the settings:

```
git config --global user.name
git config --global user.email
```

Example output:

```
Nasir Mahmood
your-email@example.com
```

**Important**
git config --global user.name does not create a GitHub account. It only configures the identity attached to your local Git commits.

## 6. Sign in to GitHub from VS Code

1. Open VS Code.
2. Look at the bottom-left area for the Accounts/profile icon.
3. Choose the option to sign in with GitHub.
4. A browser window may open.
5. Sign in to your GitHub account and authorize VS Code.
6. Return to VS Code.

After authentication, VS Code can use your GitHub account for publishing and synchronization.

## 7. Open Your PlatformIO Project

A typical ESP32 PlatformIO project may look like this:

```
ESP32_Timer/
├── include/
├── lib/
├── src/
│   └── main.cpp
├── test/
├── platformio.ini
└── .gitignore
```

The most important PlatformIO files are:

| File / Folder | Purpose |
|---|---|
| **platformio.ini** | PlatformIO project configuration. |
| **src/main.cpp** | Main ESP32 application source code. |
| **include/** | Project header files. |
| **lib/** | Project-specific libraries. |
| **test/** | PlatformIO test files. |
| **.pio/** | Generated build files; normally should NOT be uploaded to GitHub. |

## 8. Initialize Git in the PlatformIO Project

Open the VS Code terminal and make sure the terminal is inside your PlatformIO project folder.

```
git init
```

Git should report that an empty Git repository has been initialized. A hidden .git folder is created in the project.

## 9. Create the PlatformIO .gitignore (Generally already in project directory)

Create a file named:

```
.gitignore
```

A basic PlatformIO .gitignore can contain:

```
.pio/
.vscode/
```

This prevents generated build files and local VS Code settings from being unnecessarily uploaded.

**Why .pio should not be uploaded**
PlatformIO can recreate the build output on another computer. Uploading .pio can make the repository unnecessarily large and can introduce machine-specific generated files.

## 10. Stage and Commit the Project

Stage all project files:

```
git add .
```

Check what is staged:

```
git status
```

Create the first commit:

```
git commit -m "Initial ESP32 PlatformIO project"
```

A commit is a saved version of your project in Git's local history.

## 11. Create the GitHub Repository

7. Open GitHub in your web browser.
8. Choose New repository.
9. Enter a repository name, for example: ESP32_Timer.
10. Choose Public or Private according to your project needs.
11. If the project already exists locally, create the GitHub repository empty: do not add a README, .gitignore, or license at this stage.
12. Create the repository.

**Recommended naming**
Keeping the local folder name and GitHub repository name the same is not required, but it makes projects easier to recognize and manage.

## 12. Connect the Local Project to GitHub

GitHub will provide a repository URL similar to:

```
https://github.com/YOUR_USERNAME/ESP32_Timer.git
```

Example:

```
https://github.com/nasirmahmood/ESP32_Timer.git
```

Add it as the remote named origin:

```
git remote add origin https://github.com/nasirmahmood/ESP32_Timer.git
```

Verify the remote:

```
git remote -v
```

Expected pattern:

```
origin  https://github.com/nasirmahmood/ESP32_Timer.git (fetch)
origin  https://github.com/nasirmahmood/ESP32_Timer.git (push)
```

The word origin is simply the conventional name given to the remote GitHub repository.

## 13. Push the First Version to GitHub

Rename the local branch to main:

```
git branch -M main
```

Push the branch and establish the upstream relationship:

```
git push -u origin main
```

After authentication, refresh the GitHub repository page. Your PlatformIO files should appear there.

## 14. The Complete First-Time Command Sequence

For a new existing PlatformIO project, the essential sequence is:

```
git init
git config --global user.name "Nasir Mahmood"
git config --global user.email "your-email@example.com"
git add .
git commit -m "Initial ESP32 PlatformIO project"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/ESP32_Timer.git
git push -u origin main
```

**Tip**

Git configuration commands only need to be configured once per computer when using --global. The init, commit, remote, and push steps are project-specific.

## 15. Updating the Project Later

After modifying src/main.cpp or another project file, test your ESP32 project first. Then:

```
git status
git add .
git commit -m "Update ESP32 timer code"
git push
```

This is the normal day-to-day workflow:

```
Edit code
   ↓
Build / Upload / Test ESP32
   ↓
git status
   ↓
git add .
   ↓
git commit -m "Description of change"
   ↓
git push
   ↓
GitHub
```

## 16. Using the VS Code Source Control GUI

You do not have to type every Git command. VS Code provides a graphical Source Control interface.

13. Open the Source Control icon in the Activity Bar.
14. Review the changed files.
15. Click + to stage individual files, or Stage All Changes.
16. Enter a commit message.
17. Click Commit.
18. Use Sync Changes / Push to send the commit to GitHub.

## 17. Git Commands You Should Remember

| Command | Purpose |
|---|---|
| **git init** | Initialize Git in a local project. |
| **git status** | Show changed, staged, and untracked files. |
| **git add .** | Stage all changes. |
| **git commit -m "message"** | Save a version in local Git history. |
| **git remote -v** | Show the connected remote repository. |
| **git push** | Upload local commits to GitHub. |
| **git pull** | Download and integrate changes from GitHub. |

## 18. Common Problems and Solutions

**Git is not recognized:** Install Git for Windows, restart VS Code, and run git --version again.

**Wrong GitHub repository:** Run git remote -v. If necessary, replace the remote with git remote set-url origin <URL>.

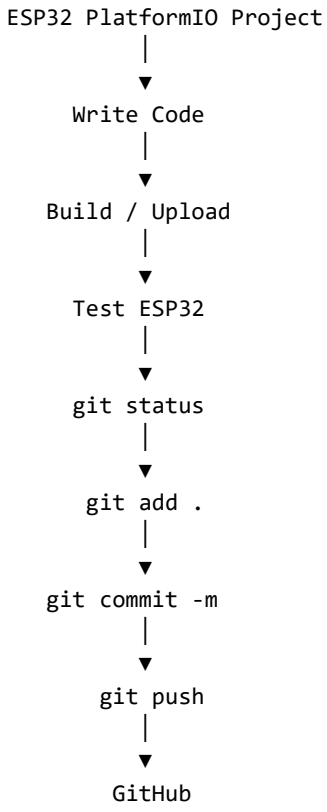
**Nothing appears on GitHub:** Run git status, make sure a commit exists, then run git push.

**Authentication problem:** Use VS Code's GitHub sign-in or Git Credential Manager. Do not place passwords directly in repository URLs.

**Large .pio folder appears:** Check .gitignore and make sure .pio/ is ignored.

**Repository already has a README:** If the remote was initialized with files, pull/merge the remote history before pushing, or recreate the repository empty for a clean first push.

## 19. Recommended ESP32 PlatformIO Git Workflow




## 20. Example Project

Suppose your GitHub username is:

```
nasirmahmood
```

and your repository is:

```
ESP32_Timer
```

Then the repository URL is:

```
https://github.com/nasirmahmood/ESP32_Timer
```

The local project can be:

```
C:\Users\YourName\Documents\PlatformIO\Projects\ESP32_Timer
```

The local project and remote repository are connected using:

```
git remote add origin https://github.com/nasirmahmood/ESP32_Timer.git
```

## 21. Quick Checklist

- ☐ Git is installed.
- ☐ Git user.name is configured.
- ☐ Git user.email is configured.

- ☐ VS Code is signed in to GitHub.
- ☐ PlatformIO project is open.
- ☐ .gitignore contains .pio/.
- ☐ git init has been executed.
- ☐ Files have been staged with git add .
- ☐ Initial commit has been created.
- ☐ GitHub repository has been created.
- ☐ origin remote points to the correct repository.
- ☐ main branch has been pushed.

## 22. Important Concepts to Remember

- Git = version-control software running on your computer.
- GitHub = online service that hosts Git repositories.
- Repository = a project managed by Git.
- Commit = a saved version/snapshot in Git history.
- Remote = a named connection to another Git repository, normally GitHub.
- origin = the conventional name for the main remote repository.
- push = send local commits to GitHub.
- pull = bring changes from GitHub into the local project.
- main = the commonly used primary branch name.

**Best practice for ESP32 projects**
Commit meaningful milestones such as 'Initial ESP32 project', 'Added MQTT communication', 'Added DHT22 sensor', or 'Fixed timer interrupt'. Avoid committing generated build output or secrets such as Wi-Fi passwords, API keys, and private credentials.

## 23. Using One GitHub Repository for the Entire Semester

For a semester-long ESP32/IoT course, it is usually convenient to maintain one GitHub repository for all laboratory tasks instead of creating a separate repository for every lab. Each lab can then be stored as a separate PlatformIO project.

Example repository name:

```
ESP32-IoT-Lab-Fall-2026
```

The recommended concept is:

```
One GitHub repository
        │
        ├── Lab 01
        ├── Lab 02
        ├── Lab 03
        ├── Lab 04
        ├── Lab 05
        └── ...
```

## 24. Recommended Semester Repository Structure

Make each laboratory task an independent PlatformIO project. This allows each lab to have its own platformio.ini, source code, libraries, and configuration.

```
ESP32-IoT-Lab-Fall-2026/
│
├── README.md
│
├── Lab01_Blink/
│   ├── platformio.ini
│   └── src/
│       └── main.cpp
│
├── Lab02_GPIO/
│   ├── platformio.ini
│   └── src/
│       └── main.cpp
│
├── Lab03_Interrupt/
│   ├── platformio.ini
│   └── src/
│       └── main.cpp
│
├── Lab04_Timer/
│   ├── platformio.ini
│   └── src/
│       └── main.cpp
│
├── Lab05_Debounce/
│   ├── platformio.ini
│   └── src/
│       └── main.cpp
│
├── Lab06_PWM/
│   ├── platformio.ini
│   └── src/
│       └── main.cpp
│
├── Lab07_ADC/
│   ├── platformio.ini
│   └── src/
│       └── main.cpp
│
├── Lab08_DHT22/
│   ├── platformio.ini
│   └── src/
│       └── main.cpp
│
└── Lab09_MQTT/
    ├── platformio.ini
    └── src/
        └── main.cpp
```

**Why this structure is useful**

Each laboratory task is independent. Students can open a particular Lab folder as a PlatformIO project, build and upload it to the ESP32, while all semester work remains organized inside one GitHub repository.

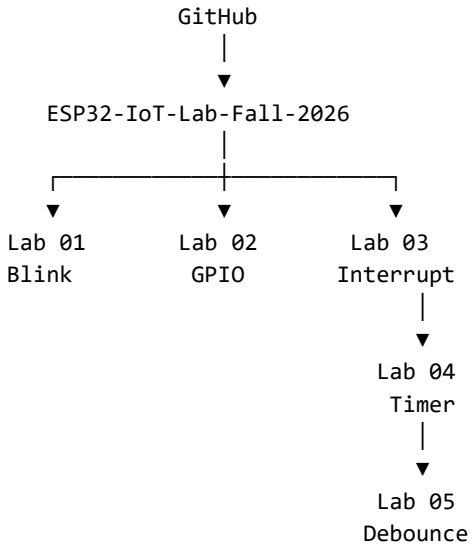
## 25. One Repository vs. Multiple Repositories

| Approach | Advantages | Disadvantages |
|---|---|---|
| One semester repository | Simple to manage; all labs in one place; easy for students and instructor to track progress. | Repository can become large if many binary/generated files are committed. |
| Separate repository per lab | Each lab is completely independent and small. | Many repositories become harder to manage, share, update, and maintain. |

**Recommended for a semester course**
Use one repository for the complete semester and separate folders for individual PlatformIO lab projects. This provides a clear progression while keeping GitHub management simple.

## 26. Semester Git Workflow

Create the GitHub repository only once at the beginning of the semester. After that, add each new laboratory task to the same repository.




## 27. First-Time Repository Setup

From the root semester folder, initialize Git:

```
git init
```

Connect it to the GitHub repository:

```
git remote add origin https://github.com/YOUR_USERNAME/ESP32-IoT-Lab-Fall-2026.git
```

Create the first commit:

```
git add .
git commit -m "Initial semester repository"
git branch -M main
git push -u origin main
```

## 28. Adding a New Lab During the Semester

Suppose Lab 04 is completed. Create or copy the Lab 04 PlatformIO project into the semester repository:

```
ESP32-IoT-Lab-Fall-2026/
└── Lab04_Timer/
    ├── platformio.ini
    └── src/
        └── main.cpp
```

Then commit and push:

```
git add .
git commit -m "Added Lab 04 Timer"
git push
```

For the next lab:

```
git add .
git commit -m "Added Lab 05 Debounce"
git push
```

## 29. Recommended Lab Naming Convention

Use a consistent naming scheme so that the repository remains easy to navigate:

```
Lab01_Blink
Lab02_GPIO
Lab03_Interrupt
Lab04_Timer
Lab05_Debounce
Lab06_PWM
Lab07_ADC
Lab08_DHT22
Lab09_MQTT
Lab10_NodeRED
```

Avoid names such as NewLab, FinalLab, MyProject, Test2, or LatestVersion because they become difficult to understand later in the semester.

## 30. Important .gitignore Rule for a Semester Repository

Because every lab is a PlatformIO project, each project may generate its own .pio directory. The root .gitignore should ignore PlatformIO build output throughout the repository.

```
.pio/
.vscode/
```

**Important**
If each lab is a separate PlatformIO project, do not manually commit its generated .pio folder. PlatformIO can recreate the build files when the project is opened and built.

## 31. Best Practice for Students

- Complete and test each lab before committing it.
- Use one meaningful commit per completed lab or major milestone.
- Use descriptive commit messages such as 'Added Lab 04 Timer'.
- Do not upload .pio build directories.
- Do not commit Wi-Fi passwords, API keys, tokens, or other secrets.
- Keep the repository structure consistent throughout the semester.
- Use the README.md file to list labs and provide short instructions.

## 32. Suggested README.md for the Semester Repository

```
# ESP32 IoT Laboratory - Fall 2026

This repository contains the ESP32 PlatformIO laboratory tasks
for the Fall 2026 semester.

## Laboratory Tasks

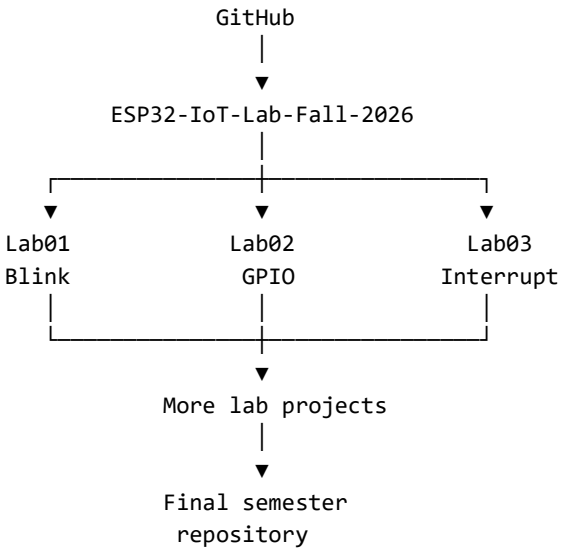
1. Lab 01 - Blink
2. Lab 02 - GPIO
3. Lab 03 - Interrupt
4. Lab 04 - Timer
5. Lab 05 - Debounce
6. Lab 06 - PWM
7. Lab 07 - ADC
8. Lab 08 - DHT22
9. Lab 09 - MQTT
10. Lab 10 - Node-RED

## Development Environment

- Visual Studio Code
- PlatformIO
- ESP32
- Arduino framework

Each laboratory is maintained as a separate PlatformIO project.
```

# 33. Final Semester Model




**Recommended model**

For a semester course, maintain one GitHub repository and put each independent ESP32/PlatformIO lab task in its own folder. This gives students a single repository to clone, while keeping every lab isolated and easy to build, test, and submit.